

Final Project

Keerthi Priya Jakku

2026-04-29

Necessary packages

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
##
## The following objects are masked from 'package:stats':
##
##   filter, lag
##
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(ggplot2)
```

```
library(caret)
```

```
## Loading required package: lattice
```

```
library(class)
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v forcats   1.0.1      v stringr   1.6.0
## v lubridate 1.9.5      v tibble   3.3.1
## v purrr     1.2.1      v tidyr    1.3.2
## v readr     2.1.6
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## x purrr::lift()    masks caret::lift()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(e1071)
```

```
##
## Attaching package: 'e1071'
##
## The following object is masked from 'package:ggplot2':
##
##   element
```

```
library(gmodels)
```

```
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Attaching package: 'pROC'
```

```
##
```

```
## The following object is masked from 'package:gmodels':
```

```
##
```

```
##      ci
```

```
##
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      cov, smooth, var
```

```
Load Data
```

```
fraudtrain_raw<-read.csv("~/Desktop/R documents/Assignment files/FML_Final project/fraudTrain.csv")
```

```
head(fraudtrain_raw)
```

```
##      X trans_date_trans_time      cc_num      merchant
## 1 0    2019-01-01 00:00:18 2.703186e+15    fraud_Rippin, Kub and Mann
## 2 1    2019-01-01 00:00:44 6.304233e+11    fraud_Heller, Gutmann and Zieme
## 3 2    2019-01-01 00:00:51 3.885949e+13    fraud_Lind-Buckridge
## 4 3    2019-01-01 00:01:16 3.534094e+15    fraud_Kutch, Hermiston and Farrell
## 5 4    2019-01-01 00:03:06 3.755342e+14    fraud_Keeling-Crist
## 6 5    2019-01-01 00:04:08 4.767265e+15    fraud_Stroman, Hudson and Erdman
##      category      amt      first      last gender      street
## 1      misc_net    4.97  Jennifer   Banks      F      561 Perry Cove
## 2      grocery_pos 107.23 Stephanie   Gill      F 43039 Riley Greens Suite 393
## 3      entertainment 220.11 Edward Sanchez      M      594 White Dale Suite 530
## 4      gas_transport 45.00 Jeremy White      M 9443 Cynthia Court Apt. 038
## 5      misc_pos    41.96 Tyler Garcia      M      408 Bradley Rest
## 6      gas_transport 94.63 Jennifer Conner      F      4655 David Island
##      city state      zip      lat      long city_pop
## 1 Moravian Falls    NC 28654 36.0788 -81.1781    3495
## 2      Orient      WA 99160 48.8878 -118.2105    149
## 3      Malad City    ID 83252 42.1808 -112.2620    4154
## 4      Boulder      MT 59632 46.2306 -112.1138    1939
## 5      Doe Hill      VA 24433 38.4207 -79.4629     99
## 6      Dublin      PA 18917 40.3750 -75.2045    2158
##      job      dob      trans_num
## 1      Psychologist, counselling 1988-03-09 0b242abb623afc578575680df30655b9
## 2      Special educational needs teacher 1978-06-21 1f76529f8574734946361c461b024d99
## 3      Nature conservation officer 1962-01-19 a1a22d70485983eac12b5b88dad1cf95
## 4      Patent attorney 1967-01-12 6b849c168bdad6f867558c3793159a81
## 5      Dance movement psychotherapist 1986-03-28 a41d7549acf90789359a9aa5346dcb46
## 6      Transport planner 1961-06-19 189a841a0a8ba03058526bcfe566aab5
##      unix_time merch_lat merch_long is_fraud
## 1 1325376018 36.01129 -82.04832 0
## 2 1325376044 49.15905 -118.18646 0
## 3 1325376051 43.15070 -112.15448 0
## 4 1325376076 47.03433 -112.56107 0
## 5 1325376186 38.67500 -78.63246 0
```

```
## 6 1325376248 40.65338 -76.15267 0
```

```
summary(fraudtrain_raw)
```

```
##           X           trans_date_trans_time           cc_num           merchant
## Min.      :      0   Length:1296675         Min.      :6.042e+10   Length:1296675
## 1st Qu.: 324168   Class :character         1st Qu.:1.800e+14   Class :character
## Median : 648337   Mode  :character         Median :3.521e+15   Mode  :character
## Mean    : 648337                                Mean    :4.172e+17
## 3rd Qu.: 972506                                3rd Qu.:4.642e+15
## Max.    :1296674                                Max.    :4.992e+18
## category          amt          first          last
## Length:1296675    Min.      :    1.00   Length:1296675   Length:1296675
## Class :character  1st Qu.:    9.65   Class :character   Class :character
## Mode  :character  Median :   47.52   Mode  :character   Mode  :character
##                               Mean    :   70.35
##                               3rd Qu.:   83.14
##                               Max.    :28948.90
## gender            street          city          state
## Length:1296675    Length:1296675   Length:1296675    Length:1296675
## Class :character  Class :character   Class :character   Class :character
## Mode  :character  Mode  :character   Mode  :character   Mode  :character
##
##
##
##           zip          lat          long          city_pop
## Min.      : 1257   Min.      :20.03   Min.      : -165.67   Min.      :    23
## 1st Qu.:26237   1st Qu.:34.62   1st Qu.: -96.80   1st Qu.:    743
## Median :48174   Median :39.35   Median : -87.48   Median :    2456
## Mean    :48801   Mean    :38.54   Mean    : -90.23   Mean    :   88824
## 3rd Qu.:72042   3rd Qu.:41.94   3rd Qu.: -80.16   3rd Qu.:   20328
## Max.    :99783   Max.    :66.69   Max.    : -67.95   Max.    :2906700
## job            dob            trans_num          unix_time
## Length:1296675   Length:1296675   Length:1296675     Min.      :1.325e+09
## Class :character  Class :character   Class :character   1st Qu.:1.339e+09
## Mode  :character  Mode  :character   Mode  :character   Median :1.349e+09
##                               Mean    :1.349e+09
##                               3rd Qu.:1.359e+09
##                               Max.    :1.372e+09
## merch_lat        merch_long        is_fraud
## Min.      :19.03   Min.      : -166.67   Min.      :0.000000
## 1st Qu.:34.73   1st Qu.: -96.90   1st Qu.:0.000000
## Median :39.37   Median : -87.44   Median :0.000000
## Mean    :38.54   Mean    : -90.23   Mean    :0.005789
## 3rd Qu.:41.96   3rd Qu.: -80.24   3rd Qu.:0.000000
## Max.    :67.51   Max.    : -66.95   Max.    :1.000000
```

```
dim(fraudtrain_raw)
```

```
## [1] 1296675      23
```

Data Preparation and Sampling

```
set.seed(123)
```

```
frauddata_sample <- fraudtrain_raw[sample(nrow(fraudtrain_raw), 10000), ]
```

A random sample of 10,000 transactions was selected from the original 1,296,675 records for computational

efficiency.

```
dim(frauddata_sample)
```

```
## [1] 10000    23
```

```
colnames(frauddata_sample)
```

```
## [1] "X"                "trans_date_trans_time" "cc_num"
## [4] "merchant"         "category"              "amt"
## [7] "first"            "last"                  "gender"
## [10] "street"           "city"                  "state"
## [13] "zip"              "lat"                   "long"
## [16] "city_pop"         "job"                   "dob"
## [19] "trans_num"        "unix_time"             "merch_lat"
## [22] "merch_long"       "is_fraud"
```

Selecting columns

```
fraud <- frauddata_sample[, c("amt", "gender", "category", "city_pop", "is_fraud")]
```

Five variables were selected that are most relevant for fraud detection — transaction amount, gender, merchant category, city population and fraud label to avoid noise.

```
Missing_Data <- colMeans(is.na(fraud))
```

```
Missing_Data
```

```
##      amt  gender category city_pop is_fraud
##      0      0      0      0      0      0
```

No missing data

Number of fraud and non fraud (Fraud=1 and non-fraud=0)

```
table(fraud$is_fraud)
```

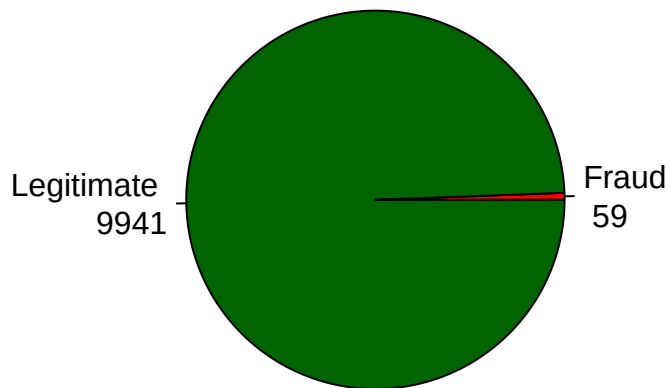
```
##
##    0    1
## 9941   59
```

Class distribution visual representation

```
fraud$is_fraud_label <- ifelse(fraud$is_fraud == 1, "Fraud", "Legitimate")
count_of_fraud <- table(fraud$is_fraud_label)
```

```
pie(count_of_fraud,
    col = c("red", "darkgreen"),
    main = "Fraud Transactions vs Legitimate Transactions",
    labels = paste(names(count_of_fraud), "\n", count_of_fraud))
```

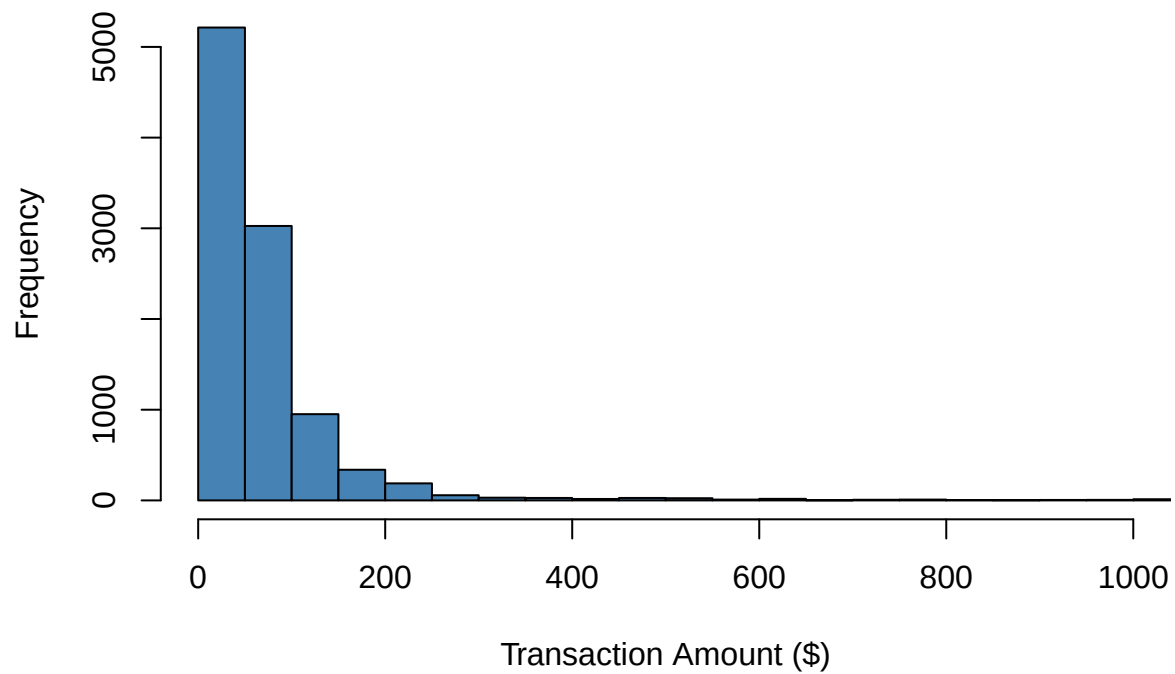
Fraud Transactions vs Legitimate Transactions



Fraud pattern using transaction amount

```
hist(fraud$amt,  
     col = "steelblue",  
     main = "Distribution of Transaction Amounts",  
     xlab = "Transaction Amount ($)",  
     breaks = 40,  
     xlim = c(0, 1000))
```

Distribution of Transaction Amounts



Transforming Data

```
fraud$gender <- ifelse(fraud$gender == "M", 1, 0)  
fraud$category <- as.numeric(as.factor(fraud$category))
```

```
fraud$is_fraud <- as.factor(fraud$is_fraud)
```

Normalizing data

```
fraud$amt <- (fraud$amt - min(fraud$amt)) / (max(fraud$amt) - min(fraud$amt))
```

```
head(fraud)
```

```
##           amt gender category city_pop is_fraud is_fraud_label
## 1237518 0.001718061      1      13   190178         0      Legitimate
## 134058  0.013917834      1       5   214703         0      Legitimate
## 1172598 0.028170805      1       7    143         0      Legitimate
## 685285  0.003320557      1      13   910148         0      Legitimate
## 1274894 0.015277644      0       6     51         0      Legitimate
## 1242203 0.069146170      0       1     922         0      Legitimate
```

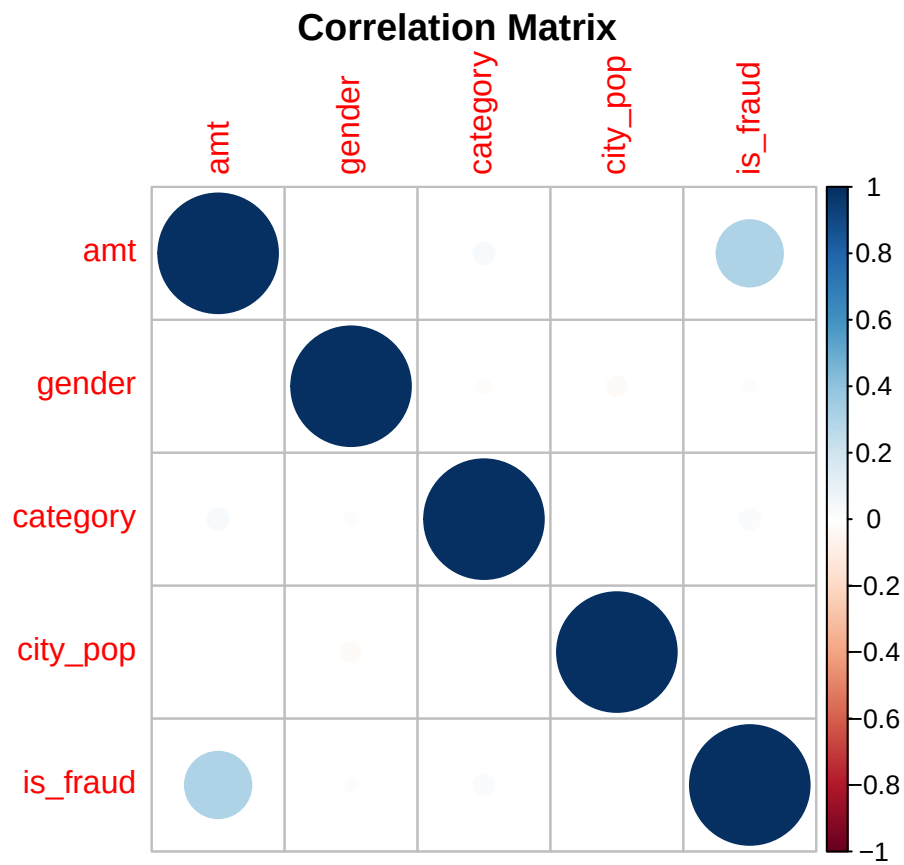
```
fraud$is_fraud_label <- NULL
```

```
fraud_correlation <- fraud
```

```
fraud_correlation$is_fraud <- as.numeric(as.character(fraud_correlation$is_fraud))
```

```
correlation_matrix <- cor(fraud_correlation)
```

```
corrplot(correlation_matrix,
  method = "circle",
  title = "Correlation Matrix",
  mar = c(0,0,1,0))
```



```
correlation_matrix
```

```
##          amt          gender          category          city_pop          is_fraud
## amt          1.000000000  0.005013763  0.031055523 -0.004749090  0.30764380
## gender      0.005013763  1.000000000 -0.012352908 -0.027903707  0.01058180
## category    0.031055523 -0.012352908  1.000000000 -0.005567361  0.02991742
## city_pop   -0.004749090 -0.027903707 -0.005567361  1.000000000  0.00578665
## is_fraud    0.307643803  0.010581803  0.029917417  0.005786650  1.00000000
```

Partitioning Data into Training (70%) and Testing (30%)

```
set.seed(123)
index_training <- createDataPartition(fraud$is_fraud, p = 0.7, list = FALSE)

training <- fraud[index_training, ]
testing <- fraud[-index_training, ]
```

KNN Model

```
training_predictors <- training[, c("amt", "gender", "category", "city_pop")]
testing_predictors <- testing[, c("amt", "gender", "category", "city_pop")]
training_labels <- training$is_fraud
testing_labels <- testing$is_fraud
```

Finding the best K

```
accuracy <- c()

for(i in 1:10){
  predict <- knn(train = training_predictors,
                 test = testing_predictors,
                 cl = training_labels,
                 k = i)
  accuracy[i] <- mean(predict == testing_labels)
}
```

```
accuracy
```

```
## [1] 0.9913304 0.9916639 0.9943314 0.9943314 0.9943314 0.9943314 0.9943314
## [8] 0.9943314 0.9943314 0.9943314
```

```
best_k <- which.max(accuracy)
best_k
```

```
## [1] 3
```

best k=3

KNN prediction using k=3

```
predicted_knn <- knn(train = training_predictors,
                    test = testing_predictors,
                    cl = training_labels,
                    k = best_k)
```

```
predicted_knn
```

```
## [1] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [38] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

[illegible]

```

## [2073] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2110] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2147] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2184] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2221] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2258] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2295] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2332] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2369] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2406] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2443] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2480] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2517] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2554] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2591] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2628] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2665] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2702] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2739] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2776] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2813] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2850] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2887] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2924] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2961] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [2998] 0 0
## Levels: 0 1

```

```

confusionMatrix(predicted_knn, testing_labels, positive = "1")

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 2982   17
##           1    0    0
##
##           Accuracy : 0.9943
##           95% CI : (0.9909, 0.9967)
##           No Information Rate : 0.9943
##           P-Value [Acc > NIR] : 0.5640232
##
##           Kappa : 0
##
## Mcnemar's Test P-Value : 0.0001042
##
##           Sensitivity : 0.000000
##           Specificity : 1.000000
##           Pos Pred Value :      NaN
##           Neg Pred Value : 0.994331
##           Prevalence : 0.005669
##           Detection Rate : 0.000000
##           Detection Prevalence : 0.000000
##           Balanced Accuracy : 0.500000
##

```

```
##          'Positive' Class : 1
##
```

Class imbalance - KNN with balanced data

```
set.seed(123)
fraud_cases      <- fraud[fraud$is_fraud == 1, ]
legitimate_cases <- fraud[fraud$is_fraud == 0, ]
legitimate_sample <- legitimate_cases[sample(nrow(legitimate_cases), 100), ]
fraud_balanced   <- rbind(fraud_cases, legitimate_sample)

index_training2 <- createDataPartition(fraud_balanced$is_fraud, p = 0.7, list = FALSE)
training2 <- fraud_balanced[index_training2, ]
testing2 <- fraud_balanced[-index_training2, ]

training_predictors2 <- training2[, c("amt", "gender", "category", "city_pop")]
testing_predictors2 <- testing2[, c("amt", "gender", "category", "city_pop")]
training_labels2 <- training2$is_fraud
testing_labels2 <- testing2$is_fraud
accuracy <- c()

accuracy2 <- c()

for(i in 1:10){
  pred2 <- knn(train = training_predictors2,
               test  = testing_predictors2,
               cl    = training_labels2,
               k      = i)
  accuracy2[i] <- mean(pred2 == testing_labels2)
}

accuracy2

## [1] 0.5531915 0.6382979 0.6170213 0.6170213 0.5957447 0.6808511 0.6595745
## [8] 0.5744681 0.5531915 0.6382979

best_k2 <- which.max(accuracy2)
best_k2
```

```
## [1] 6
```

```
predicted_knn2 <- knn(train = training_predictors2,
                     test  = testing_predictors2,
                     cl    = training_labels2,
                     k      = best_k2)
```

Confusion matrix with balanced data

```
confusionMatrix(predicted_knn2, testing_labels2, positive = "1")
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction 0 1
```

```
##           0 23 12
```

```
##          1  7  5
##
##          Accuracy : 0.5957
##          95% CI : (0.4427, 0.7363)
##    No Information Rate : 0.6383
##    P-Value [Acc > NIR] : 0.7777
##
##          Kappa : 0.0649
##
##    McNemar's Test P-Value : 0.3588
##
##          Sensitivity : 0.2941
##          Specificity : 0.7667
##    Pos Pred Value : 0.4167
##    Neg Pred Value : 0.6571
##          Prevalence : 0.3617
##    Detection Rate : 0.1064
##    Detection Prevalence : 0.2553
##    Balanced Accuracy : 0.5304
##
##    'Positive' Class : 1
##
```

Naive Bayes approach

```
naivebayes_model <- naiveBayes(is_fraud ~ ., data = training)
naivebayes_model
```

```
##
## Naive Bayes Classifier for Discrete Predictors
##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace)
##
## A-priori probabilities:
## Y
##          0          1
## 0.994000857 0.005999143
##
## Conditional probabilities:
##      amt
## Y      [,1]      [,2]
## 0 0.0251082 0.04142065
## 1 0.2056084 0.15707221
##
##      gender
## Y      [,1]      [,2]
## 0 0.4579681 0.4982660
## 1 0.5476190 0.5037605
##
##      category
## Y      [,1]      [,2]
## 0 7.158931 3.917851
## 1 8.738095 3.920301
##
```

Prediction using Naive Bayes

[illegible]


```
##                Kappa : 0.2366
##
## Mcnemar's Test P-Value : 1.872e-07
##
##          Sensitivity : 0.529412
##          Specificity : 0.983903
##          Pos Pred Value : 0.157895
##          Neg Pred Value : 0.997281
##          Prevalence : 0.005669
##          Detection Rate : 0.003001
##          Detection Prevalence : 0.019006
##          Balanced Accuracy : 0.756658
##
##          'Positive' Class : 1
##
```

```
predicted_nb_probability <- predict(naivebayes_model, testing, type = "raw")
```

```
head(predicted_nb_probability)
```

```
##           0           1
## [1,] 0.9994040 0.0005959581
## [2,] 0.9992351 0.0007648626
## [3,] 0.9990419 0.0009580507
## [4,] 0.9989855 0.0010145489
## [5,] 0.9985939 0.0014060879
## [6,] 0.9991508 0.0008492086
```

Model Performance Comparison Using ROC Curve

```
roc_naivebayes <- roc(testing$is_fraud, predicted_nb_probability[, "1"])
```

```
## Setting levels: control = 0, case = 1
```

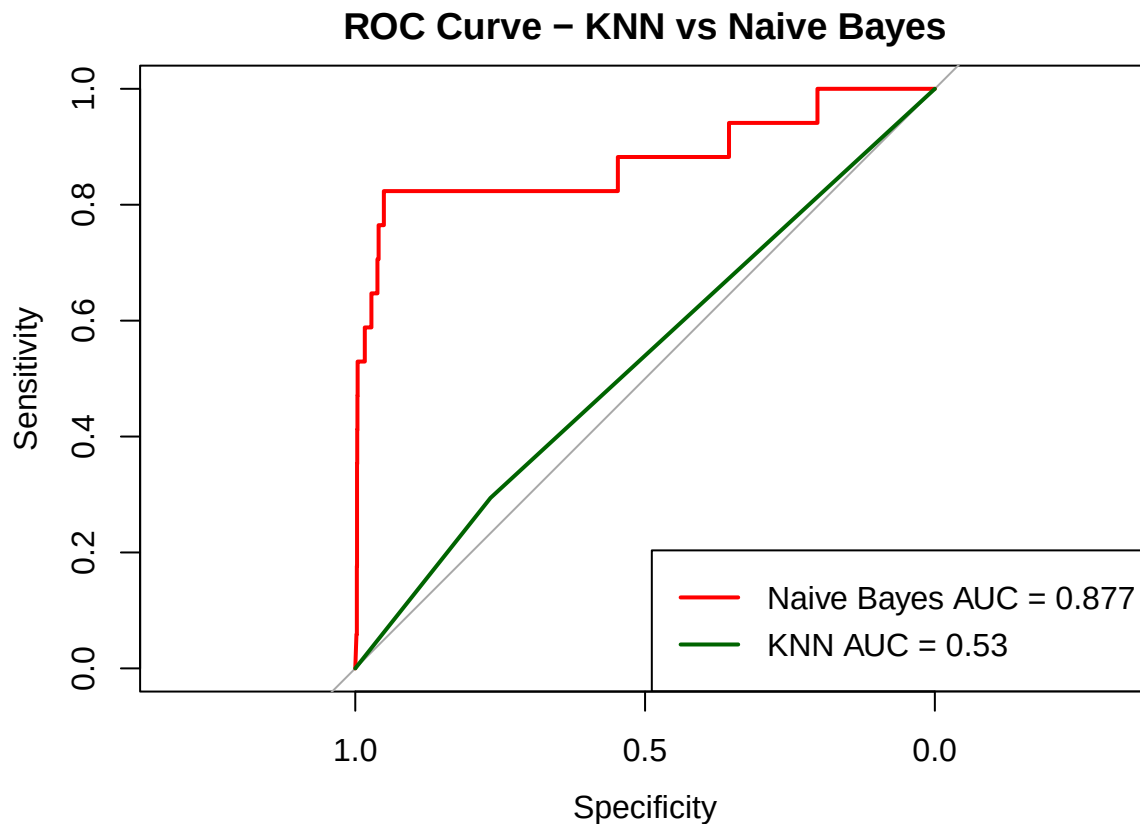
```
## Setting direction: controls < cases
```

```
roc_knn <- roc(testing_labels2, as.numeric(as.character(predicted_knn2)))
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
plot(roc_naivebayes, col = "red", main = "ROC Curve - KNN vs Naive Bayes")
plot(roc_knn, col = "darkgreen", add = TRUE)
legend("bottomright",
      legend = c(paste("Naive Bayes AUC =", round(auc(roc_naivebayes), 3)),
                 paste("KNN AUC =", round(auc(roc_knn), 3))),
      col = c("red", "darkgreen"), lwd = 2)
```



Models Comparision table

```
comparison <- data.frame(
  Model = c("KNN", "Naive Bayes"),
  Accuracy= c("59.57%", "98.13%"),
  Sensitivity= c("29.41%", "52.94%"),
  Specificity = c("76.67%", "98.39%"),
  AUC= c("0.530", "0.877")
)
```

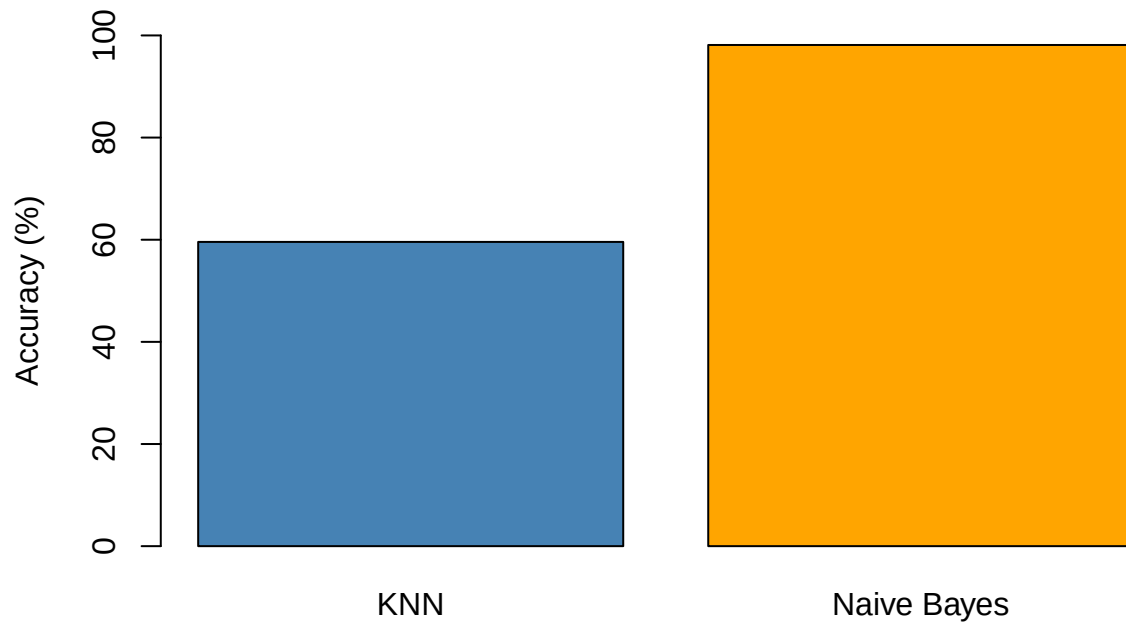
comparison

##	Model	Accuracy	Sensitivity	Specificity	AUC
## 1	KNN	59.57%	29.41%	76.67%	0.530
## 2	Naive Bayes	98.13%	52.94%	98.39%	0.877

Visulalization of the Comparision

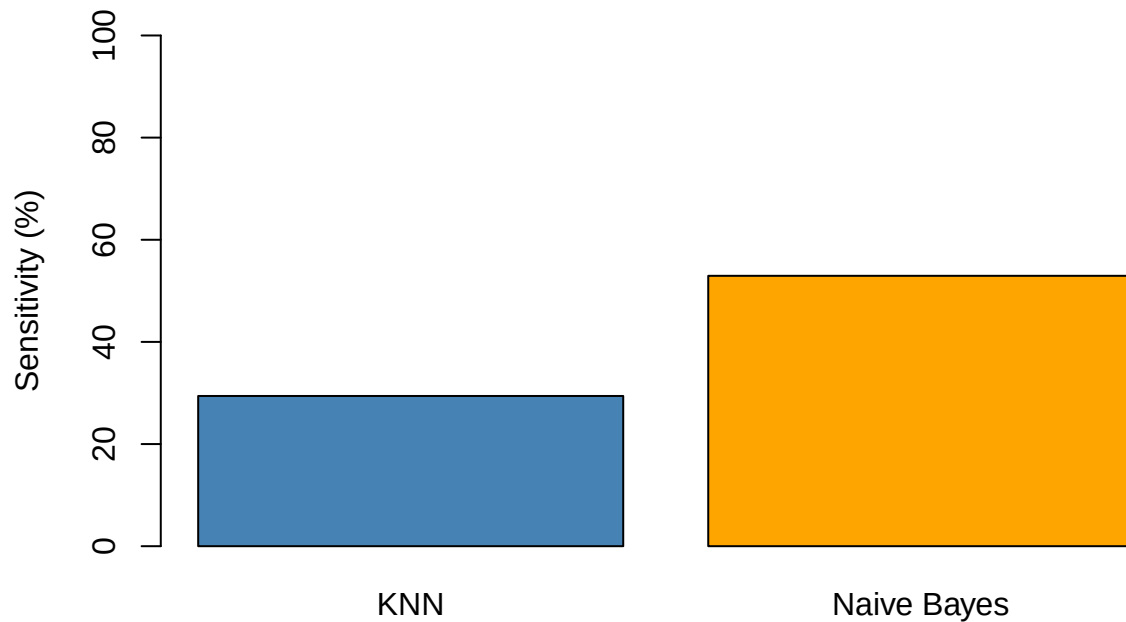
```
barplot(c(59.57, 98.13),
  names.arg = c("KNN", "Naive Bayes"),
  col = c("steelblue", "orange"),
  main = "Accuracy Comparison",
  ylab = "Accuracy (%)",
  ylim = c(0, 100))
```

Accuracy Comparison



```
barplot(c(29.41, 52.94),  
        names.arg = c("KNN", "Naive Bayes"),  
        col = c("steelblue", "orange"),  
        main = "Sensitivity Comparison",  
        ylab = "Sensitivity (%)",  
        ylim = c(0, 100))
```

Sensitivity Comparison



Analysis:

1. The original dataset contains 1,296,675 real credit card transactions. Due to the computational limitations of processing such a large dataset in R, a random sample of 10,000 transactions were taken for the analysis.

Only 59 out of 10,000 transactions are fraudulent (0.59%), confirming the severe class imbalance typical of real world fraud data. The pie chart confirms the class imbalance, legitimate transactions dominating the dataset.

2. Most transactions are of small amounts with very few exceeding \$500.

3. Transaction amount was normalized to a 0-1 scale to prevent it from dominating the KNN model. The amounts show a strongest correlation with fraud compared to other variables.

4. The class imbalance caused the KNN to predict all transactions to be legitimate and failed to detect fraud. To address the imbalance, a balanced dataset was created using all 59 fraudulent and 100 randomly sampled legitimate transactions.

5. Naive Bayes was applied to the full 10,000 transaction dataset without requiring balancing and achieved an accuracy of 98.13% and sensitivity of 52.94%, successfully identifying 9 out of 17 fraudulent transactions. This approach also provides a fraud probability score for each transaction, which can be used by banks to prioritize fraud investigations.

6. Naive Bayes AUC of 0.877 significantly outperforms KNN AUC of 0.530, confirming Naive Bayes as the superior model outperforming KNN across all metrics — accuracy, sensitivity, specificity and AUC.

Findings: The aim is to detect fraudulent credit card transactions using the KNN and Naive Bayes Machine learning techniques. The severe class imbalance presented a significant challenge, particularly for KNN, which failed to detect any fraud when applied to the full dataset. After addressing the imbalance through undersampling, KNN achieved an accuracy of 59.57% and a sensitivity of 29.41%, identifying only 5 out of 17 fraudulent transactions in the testing set. Naive Bayes was applied directly to the full 10,000 transaction dataset without requiring any balancing and achieved a significantly higher accuracy of 98.13% and sensitivity of 52.94%, successfully identifying 9 out of 17 fraudulent transactions. The ROC curve further confirmed Naive Bayes as the superior model.

Based on these findings, Naive Bayes is recommended as the more effective model for real world credit card fraud detection, as it handles class imbalance naturally and provides probability scores that banks can use to prioritize fraud investigations and protect customers from financial losses.